torch.func.functional_call#

https://pytorch.org/docs/stable/generated/torch.func.functional_call.html

Description:

Performs a functional call on the module by replacing the module parameters and buffers with the provided ones. If the module has active parametrizations, passing a value in the parameter_and_buffer_dicts argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as {submodule_name}.parametrizations.{parameter_name}.original. If the module performs in-place operations on parameters/buffers, these will be reflected in the parameter_and_buffer_dicts input.

Function Signature

```
torch.func.functional_call(module,
parameter_and_buffer_dicts, args=None, kwargs=None, *,
tie_weights=True, strict=False)[source]#

Parameters

Returns

Return type
```

Returns:

Any

Code Examples

```
>>> a = {'foo': torch.zeros(())}
>>> mod = Foo()  # does self.foo = self.foo + 1
>>> print(mod.foo)  # tensor(0.)
>>> functional_call(mod, a, torch.ones(()))
>>> print(mod.foo)  # tensor(0.)
>>> print(a['foo'])  # tensor(1.)
```

```
>>> a = {'foo': torch.zeros(())}
>>> mod = Foo()  # has both self.foo and self.foo_tied which are
tied. Returns x + self.foo + self.foo_tied
>>> print(mod.foo)  # tensor(1.)
>>> mod(torch.zeros(()))  # tensor(2.)
>>> functional_call(mod, a, torch.zeros(()))  # tensor(0.) since
it will change self.foo_tied too
>>> functional_call(mod, a, torch.zeros(()), tie_weights=False)
# tensor(1.)--self.foo_tied is not updated
>>> new_a = {'foo': torch.zeros(()), 'foo_tied':
torch.zeros(())}
>>> functional_call(mod, new_a, torch.zeros()) # tensor(0.)
```

```
a = (
    {"weight": torch.ones(1, 1)},
    {"buffer": torch.zeros(1)},
)  # two separate dictionaries
mod = nn.Bar(1, 1)  # return self.weight @ x + self.buffer
print(mod.weight)  # tensor(...)
print(mod.buffer)  # tensor(...)
x = torch.randn((1, 1))
print(x)
functional_call(mod, a, x)  # same as x
print(mod.weight)  # same as before functional_call
```

```
import torch
import torch.nn as nn
from torch.func import functional_call, grad

x = torch.randn(4, 3)
t = torch.randn(4, 3)
model = nn.Linear(3, 3)


def compute_loss(params, x, t):
    y = functional_call(model, params, x)
    return nn.functional.mse_loss(y, t)


grad_weights = grad(compute_loss)
(dict(model.named_parameters()), x, t)
```

```
>>> detached_params = {k: v.detach() for k, v in
model.named_parameters()}
>>> grad_weights = grad(compute_loss)(detached_params, x, t)
>>> grad_weights.grad_fn  # None--it's not tracking gradients
outside of grad
```

Notes & Warnings

NOTE Note If the module has active parametrizations, passing a value in the parameter_and_buffer_dicts argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as {submodule_name}.parametrizations.{parameter_name}.original.

NOTE Note If the module performs in-place operations on parameters/buffers, these will be reflected in the parameter_and_buffer_dicts input. Example: >>> a = {'foo': torch.zeros(())} >>> mod = Foo() # does self.foo = self.foo + 1 >>> print(mod.foo) # tensor(0.) >>> functional_call(mod, a, torch.ones(())) >>> print(mod.foo) # tensor(0.) >>> print(a['foo']) # tensor(1.)

NOTE Note If the module has tied weights, whether or not functional_call respects the tying is determined by the tie_weights flag. Example: >>> a = {'foo': torch.zeros(())} >>> mod = Foo() # has both self.foo and self.foo_tied which are tied. Returns x + self.foo + self.foo_tied >>> print(mod.foo) # tensor(1.) >>> mod(torch.zeros(())) # tensor(2.) >>> functional_call(mod, a, torch.zeros(())) # tensor(0.) since it will change self.foo_tied too >>> functional_call(mod, a, torch.zeros(()), tie_weights=False) # tensor(1.)--self.foo_tied is not updated >>> new_a = {'foo': torch.zeros(()), 'foo_tied': torch.zeros(())} >>> functional_call(mod, new_a, torch.zeros()) # tensor(0.)

NOTE

Note If the user does not need grad tracking outside of grad transforms, they can detach all of the parameters for better performance and memory usage Example: >>> detached_params = {k: v.detach() for k, v in model.named_parameters()} >>> grad_weights = grad(compute_loss)(detached_params, x, t) >>> grad_weights.grad_fn # None--it's not tracking gradients outside of grad This means that the user cannot call grad_weight.backward(). However, if they don’t need autograd tracking outside of the transforms, this will result in less memory usage and faster speeds.

Detailed Documentation

Documentation

Performs a functional call on the module by replacing the module parameters and buffers with the provided ones.

If the module has active parametrizations, passing a value in the parameter_and_buffer_dicts argument with the name set to the regular parameter name will completely disable the parametrization. If you want to apply the parametrization function to the value passed please set the key as {submodule_name}.parametrizations.{parameter_name}.original.

If the module performs in-place operations on parameters/buffers, these will be reflected in the parameter_and_buffer_dicts input.

If the module has tied weights, whether or not functional_call respects the tying is determined by the tie_weights flag.

An example of passing multiple dictionaries

And here is an example of applying the grad transform over the parameters of a model.

If the user does not need grad tracking outside of grad transforms, they can detach all of the parameters for better performance and memory usage

This means that the user cannot call grad_weight.backward(). However, if they don't need autograd tracking outside of the transforms, this will result in less memory usage and faster speeds.

module (torch.nn.Module) – the module to call

parameters_and_buffer_dicts (Dict[str, Tensor] or tuple of Dict[str, Tensor]) – the parameters that will be used in the module call. If given a tuple of dictionaries, they must have distinct keys so that all dictionaries can be used together

args (Any or tuple) – arguments to be passed to the module call. If not a tuple, considered a single argument.

kwargs (dict) – keyword arguments to be passed to the module call

tie_weights (bool, optional) – If True, then parameters and buffers tied in the original model will be treated as tied in the reparameterized version. Therefore, if True and different values are passed for the tied parameters and buffers, it will error. If False, it will not respect the originally tied parameters and buffers unless the values passed for both weights are the same. Default: True.

strict (bool, optional) – If True, then the parameters and buffers passed in must match the parameters and buffers in the original module. Therefore, if True and there are any missing or unexpected keys, it will error. Default: False.

the result of calling module.